Episteme

A research intelligence system that builds a living knowledge graph from scientific literature — powered by GraphRAG, semantic search, fine-tuned NLP models, and agentic reasoning.

» submit a suggestion · » report a bug · » contact

Forks 0 Stars 0

CI passing | pull requests 0 open | last commit today | license Apache-2.0 | contributions welcome

🕹️ Table of Content

1. About
   - Tech Stack
   - Features
2. Documentation
   - Setup
   - Development
3. Contributing
4. License

🌻 About

Episteme (from ἐπιστήμη) is a research intelligence system built over scientific literature. It ingests papers from arXiv and Semantic Scholar, extracts scientific entities and relationships using fine-tuned NLP models, and constructs a knowledge graph that connects concepts, methods, authors, datasets, and citations across papers. On top of that graph, it layers hybrid semantic search and a LangGraph-powered agent capable of answering multi-step research questions with citations and reasoning traces.

The project implements full ML engineering stack for NLP systems including data ingestion pipelines, information extraction, retrieval-augmented generation, knowledge graph construction, LLM fine-tuning, agent orchestration, evaluation, and a fullstack interface.

:hammer_and_wrench: Tech Stack

➕ Development Tools

- ☑️ Python 3.11+
- ☑️ Poetry (dependency management)
- ☑️ Jupyter Notebooks (experimentation)
- ☑️ VSCode

➕ Data & NLP

- ☑️ arXiv API (paper ingestion)

- ✅ Semantic Scholar API (metadata + citations)
- ✅ PyMuPDF (PDF parsing)
- ✅ spaCy (NLP baseline)
- ✅ HuggingFace Transformers (fine-tuning + NER)
- ✅ SciSpaCy (scientific NLP models)
- ✅ sentence-transformers (embeddings)

➕ Storage & Retrieval

- ✅ ChromaDB (vector store)
- ✅ Neo4j Community Edition (knowledge graph)
- ✅ SQLite (metadata store)

➕ ML & Agents

- ✅ Ollama + Llama 3 (local LLM inference)
- ✅ LangChain + LangGraph (agent orchestration)
- ✅ QLoRA via HuggingFace PEFT (fine-tuning)
- ✅ RAGAS (RAG evaluation framework)

➕ Backend

- ✅ FastAPI
- ✅ Pydantic

➕ Frontend

- ✅ React
- ✅ D3.js / Cytoscape.js (graph visualization)

➕ DevOps

- ✅ GitHub Actions (CI)
- ✅ Docker (containerization)

🍄 Features

➕ Paper Ingestion Pipeline

- ✅ Fetch papers by keyword, topic, or author from arXiv and Semantic Scholar
- ✅ Parse PDFs into structured sections (abstract, introduction, methods, results)
- ✅ Chunk and preprocess text for downstream NLP

➕ Scientific Entity & Relationship Extraction

- ✅ Fine-tuned NER model for scientific entities (methods, datasets, concepts, metrics)
- ✅ Relationship extraction between entities across sentences and papers

- ✅ Structured output pipeline feeding directly into the knowledge graph

➕ Knowledge Graph Construction

- ✅ Neo4j graph of papers, authors, concepts, methods, datasets, institutions, and citations
- ✅ Graph traversal queries for concept lineage, author networks, and method evolution
- ✅ Visual graph explorer in the frontend

➕ Hybrid Semantic Search

- ✅ Dense retrieval via fine-tuned domain embeddings model
- ✅ Sparse retrieval via BM25
- ✅ Reranking layer for result quality
- ✅ Search over full papers, sections, or entities independently

➕ GraphRAG Query Layer

- ✅ Combines vector retrieval with graph traversal for context-aware answers
- ✅ Citations and source attribution on every generated response
- ✅ Explainable retrieval — shows which nodes and edges informed the answer

➕ LangGraph Research Agent

- ✅ Multi-step agent that plans, searches, traverses the graph, and synthesizes answers
- ✅ Handles compound research questions ("what methods address X, and who pioneered them?")
- ✅ Reasoning trace visible in the frontend

➕ Fine-Tuned Models

- ✅ Domain-adapted sentence-transformer for scientific embeddings
- ✅ Fine-tuned NER model on SciERC dataset for scientific entity recognition
- ✅ Benchmark comparison between base and fine-tuned models

➕ Evaluation Harness

- ✅ RAGAS metrics (faithfulness, answer relevancy, context recall, context precision)
- ✅ NER model benchmarks (precision, recall, F1 per entity type)
- ✅ Retrieval quality metrics (MRR, NDCG)

🌵 Documentation

🍯 Setup

```
# clone the repo
git clone https://github.com/pallas-lab/episteme.git
cd episteme
```

```
# install python dependencies
poetry install

# copy environment variables
cp .env.example .env

# start Neo4j (via Docker)
docker-compose up neo4j -d

# start Ollama and pull the model
ollama pull llama3

# start the backend
poetry run uvicorn backend.app.main:app --reload

# install frontend dependencies and start
cd frontend
npm install
npm run dev
```

🍎 Development

- Milestone 0 — Project Scaffold
- Milestone 1 — Data Ingestion Pipeline
- Milestone 2 — NLP Extraction + Knowledge Graph
- Milestone 3 — Embeddings + Vector Store
- Milestone 4 — Hybrid Semantic Search
- Milestone 5 — GraphRAG Query Layer
- Milestone 6 — LangGraph Research Agent
- Milestone 7 — Fine-Tuning (Embeddings + NER)
- Milestone 8 — FastAPI Backend
- Milestone 9 — React Frontend
- Milestone 10 — Evaluation Harness

🌾 Contributing

> 1. Fork the Project
> 2. Create your Branch (`git checkout -b my-branch`)
> 3. Commit your Changes (`git commit -m 'add my contribution'`)
> 4. Push to the Branch (`git push --set-upstream origin my-branch`)
> 5. Open a Pull Request

📝 License

This project is licensed under the Apache License 2.0.